M12 # Hooks: Concrete Patterns

Methodology Intermediate

Practical hook examples to automate your workflow

Auto-format after editing

`PostToolUse`, matcher `Edit|Write`: format by extension.

```
INPUT=$(cat)
FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path')
EXT="${FILE##*.}"
case "$EXT" in
  ts|tsx) npx prettier --write "$FILE" 2>/dev/null ;;
  py)     black "$FILE" 2>/dev/null ;;
  go)     gofmt -w "$FILE" 2>/dev/null ;;
  rs)     rustfmt "$FILE" 2>/dev/null ;;
esac
exit 0
```

Wait for formatting before testing or editing again to avoid concurrent writes.

macOS notification on completion

`Stop`: show a macOS notification and play a sound.

```
INPUT=$(cat)
MESSAGE=$(echo "$INPUT" | jq -r '.last_assistant_message // "Done"')
osascript - "$MESSAGE" <<'APPLESCRIPT'
on run argv
  display notification (item 1 of argv) with title "Claude Code"
end run
APPLESCRIPT
afplay /System/Library/Sounds/Hero.aiff &
exit 0
```

Security: block dangerous commands

`PreToolUse`, matcher `Bash`: inspect the proposed command.

```
COMMAND=$(cat | jq -r '.tool_input.command')
DANGEROUS=("rm -rf /" "dd if=" "DROP DATABASE"
           "git push --force.*main")
for pattern in "${DANGEROUS[@]}"; do
  if [[ "$COMMAND" =~ $pattern ]]; then
    echo "BLOCKED: $pattern" >&2
    exit 2
  fi
done
exit 0
```

Exit 2 blocks this action. These illustrative patterns are bypassable; retain permissions and isolation.

Git auto-stage after editing

`PostToolUse`, matcher `Edit|Write`. Opt-in staging; review the index before committing.

```
INPUT=$(cat)
FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')
[[ -z "$FILE" || "$FILE" == "null" ]] && exit 0
git add -- "$FILE" 2>/dev/null
exit 0
```

RTK auto-wrapper

For `PreToolUse` on `Bash`, suggest an RTK equivalent through JSON `hookSpecificOutput.additionalContext`. Plain stdout is not a reliable model instruction. Verify wrapper support before rewriting a command.

Log all executed commands

`PostToolUse`, matcher `Bash`: this example creates its own log directory.

```
COMMAND=$(cat | jq -r '.tool_input.command // empty')
mkdir -p ~/.claude/logs
printf '%s | %s\n' "$(date -u +%FT%T)" "$COMMAND" \
  >> ~/.claude/logs/commands.log
exit 0
```

Dispatcher pattern: a single entry point

Use one `Edit|Write|Bash` matcher and route by tool name to `.claude/hooks/handlers/`. Preserve each handler's decision and exit status.

Tool output replacement (v2.1.121)

`PostToolUse`: `hookSpecificOutput.updatedToolOutput` replaces the result delivered to Claude. Match the actual response schema.

```
jq 'select(.tool_name == "Bash") |
  {hookSpecificOutput: {hookEventName: "PostToolUse",
    updatedToolOutput: (.tool_response |
      .stdout |= gsub("ghp_[A-Za-z0-9]+"; "[REDACTED]") |
      .stderr |= gsub("ghp_[A-Za-z0-9]+"; "[REDACTED]"))}}'
```

This covers one token pattern. It cannot undo side effects or redact earlier telemetry.

Installation checklist

```
chmod +x .claude/hooks/my-hook.sh   # Required
# Test manually before activating:
echo '{"tool_name":"Edit","tool_input":{"file_path":"test.ts"}}' \
  | .claude/hooks/my-hook.sh
```